

Assignment-1

Date: 29 Aug 2025
Course: CS5013 GPU Programming
Name: Lochan Surya Teja Neeli
Roll No: CS25MTECH11015

Experimental Setup

CPU Details

```
$lscpu
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 46 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 40
On-line CPU(s) list: 0-39
Vendor ID: GenuineIntel
Model name: Intel(R) Xeon(R) Silver 4114 CPU @ 2.20GHz
CPU family: 6
Model: 85
Thread(s) per core: 2
Core(s) per socket: 10
Socket(s): 2
Stepping: 4
CPU max MHz: 3000.0000
CPU min MHz: 800.0000
```

1. Datatype used: int32
2. Ran on the GPU: NVIDIA Tesla-P100
3. This report consists of experimental results/insights majorly for 64×64 matrices

GPU Details(Device Query)

```
./deviceQuery Starting...

CUDA Device Query (Runtime API) version (CUDA static linking)

Detected 2 CUDA Capable device(s)

Device 0: "Tesla P100-PCIE-16GB"
  CUDA Driver Version / Runtime Version 12.8 / 12.6
  CUDA Capability Major/Minor version number: 6.0
  Total amount of global memory: 16269 MBytes (17059545088 bytes)
  (056) Multiprocessors, (064) CUDA Cores/MP: 3584 CUDA Cores
  GPU Max Clock rate: 1329 MHz (1.33 GHz)
  Memory Clock rate: 715 Mhz
  Memory Bus Width: 4096-bit
  L2 Cache Size: 4194304 bytes
  Maximum Texture Dimension Size (x,y,z) 1D=(131072), 2D=(131072,
                                         65536), 3D=(16384, 16384,
                                         16384)
  Maximum Layered 1D Texture Size, (num) layers 1D=(32768), 2048
    layers
  Maximum Layered 2D Texture Size, (num) layers 2D=(32768, 32768),
    2048 layers
  Total amount of constant memory: 65536 bytes
  Total amount of shared memory per block: 49152 bytes
```

```

Total shared memory per multiprocessor: 65536 bytes
Total number of registers available per block: 65536
Warp size: 32
Maximum number of threads per multiprocessor: 2048
Maximum number of threads per block: 1024
Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
Max dimension size of a grid size (x,y,z): (2147483647, 65535,
65535)
Maximum memory pitch: 2147483647 bytes
Texture alignment: 512 bytes
Concurrent copy and kernel execution: Yes with 2 copy engine(s)
Run time limit on kernels: No
Integrated GPU sharing Host Memory: No
Support host page-locked memory mapping: Yes
Alignment requirement for Surfaces: Yes
Device has ECC support: Enabled
Device supports Unified Addressing (UVA): Yes
Device supports Managed Memory: Yes
Device supports Compute Preemption: Yes
Supports Cooperative Kernel Launch: Yes
Supports MultiDevice Co-op Kernel Launch: Yes
Device PCI Domain ID / Bus ID / location ID: 0 / 55 / 0
Compute Mode:
    < Default (multiple host threads can use ::cudaSetDevice() with
    device simultaneously) >

```

```

Device 1: "Tesla P100-PCIE-16GB"
CUDA Driver Version / Runtime Version 12.8 / 12.6
CUDA Capability Major/Minor version number: 6.0
Total amount of global memory: 16269 MBytes (17059545088 bytes)
(056) Multiprocessors, (064) CUDA Cores/MP: 3584 CUDA Cores
GPU Max Clock rate: 1329 MHz (1.33 GHz)
Memory Clock rate: 715 Mhz
Memory Bus Width: 4096-bit
L2 Cache Size: 4194304 bytes
Maximum Texture Dimension Size (x,y,z) 1D=(131072), 2D=(131072,
65536), 3D=(16384, 16384,
16384)
Maximum Layered 1D Texture Size, (num) layers 1D=(32768), 2048
layers
Maximum Layered 2D Texture Size, (num) layers 2D=(32768, 32768),
2048 layers
Total amount of constant memory: 65536 bytes
Total amount of shared memory per block: 49152 bytes
Total shared memory per multiprocessor: 65536 bytes
Total number of registers available per block: 65536
Warp size: 32
Maximum number of threads per multiprocessor: 2048
Maximum number of threads per block: 1024
Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
Max dimension size of a grid size (x,y,z): (2147483647, 65535,
65535)
Maximum memory pitch: 2147483647 bytes
Texture alignment: 512 bytes
Concurrent copy and kernel execution: Yes with 2 copy engine(s)
Run time limit on kernels: No
Integrated GPU sharing Host Memory: No
Support host page-locked memory mapping: Yes
Alignment requirement for Surfaces: Yes
Device has ECC support: Enabled
Device supports Unified Addressing (UVA): Yes
Device supports Managed Memory: Yes
Device supports Compute Preemption: Yes
Supports Cooperative Kernel Launch: Yes
Supports MultiDevice Co-op Kernel Launch: Yes
Device PCI Domain ID / Bus ID / location ID: 0 / 134 / 0
Compute Mode:
    < Default (multiple host threads can use ::cudaSetDevice() with
    device simultaneously) >

```

```

> Peer access from Tesla P100-PCIE-16GB (GPU0) -> Tesla P100-PCIE
-16GB (GPU1) : Yes
> Peer access from Tesla P100-PCIE-16GB (GPU1) -> Tesla P100-PCIE
-16GB (GPU0) : Yes

deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 12.8, CUDA
Runtime Version = 12.6,
NumDevs = 2
Result = PASS

```

Important Device Parameters to Consider for Evaluating the Performance of the GPU Device Kernel:

1. **Total Amount Global Memory:** 16GB
2. **Number of SM(Streaming Multiprocessor)s:** 56
3. **Number of Blocks per SM:** 2
4. **Number of CUDA Cores(Actual ALU of GPU) per SM:** 64
5. **Register File Size(per block):** 65536
6. **Maximum Block Size:** 1024 threads
7. **Maximum number of threads per SM:** 2048 $\Rightarrow$ 2 Blocks
8. **Maximum Amount of Shared Memory per Block:** 48KB(49512B)
9. **Memory Bus Width(Band Width Indicator):** 4KB(4096B)

Factors Limiting the Performance of a Function on a Processor

1. **Memory Bandwidth:** Bytes Per Second
2. **Math Bandwidth**(e.g., FMA(Fused Multiply- Add)) FLOPS
3. **Latency:** “the time from when a request is made for a transaction or operation, until that operation completes” [1].

Memory Coalescing Rules

”memory operations issued from separate instructions never coalesce with each other. coalescing is a behavior that applies to memory activity emanating from a single instruction.” [2]

Latency Hiding

The delay between threads making a request from shared memory and getting data may not be a problem at all even when there are bank conflicts.

If there are many threads running on an SM, the scheduler can simply switch to another warp while the banks figure out the requested data.

The latency of worst possible request pattern might be completely hidden and irrelevant.

The only real way to know if bank conflicts are slowing an algorithm is to experiment with padding and access patterns.

Inter-Block Conflicts?

Threads in different blocks do not cause bank conflicts with each other. Bank Conflicts are only an important consideration on the **warp-level**, not the blocks. For instance, if you have a kernel allocated 32 floats of shared memory, then the second block's shared memory will be placed directly after the first. But the two blocks will have no trouble reading from shared memory at a time.

1.1. Matrix-Matrix Multiplication using 1D Grid Parameters

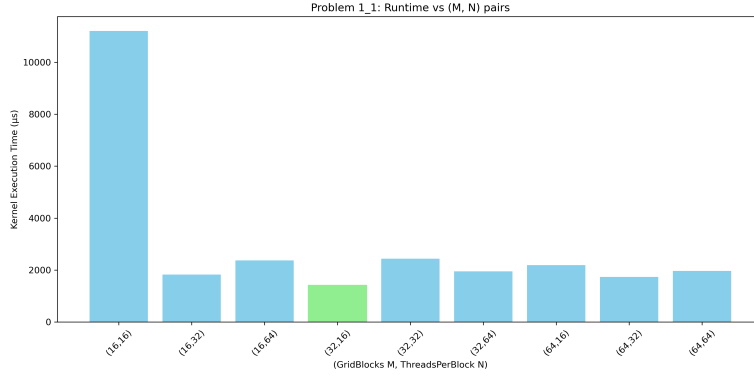
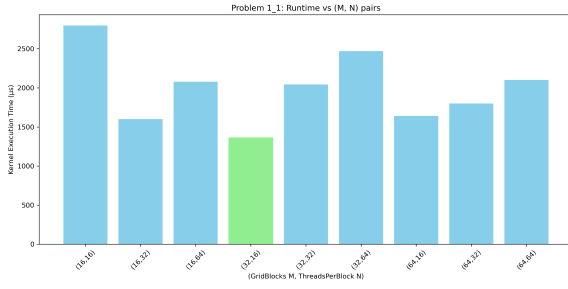


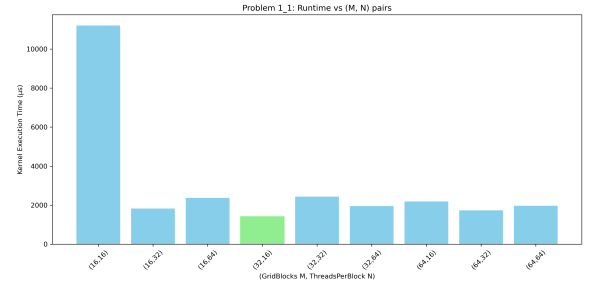
Figure 1: matmul_1d performance for 64×64 Matrix

Trial No.	#Thread	Blocks(M)	Block Size(N)	Kernel exec. Time (μ s)	Insights
1	16	16	16	2797	Thread Coarsening: Total Number of threads launched are lesser than the number of entries in the input data points;
2	16	32	32	1600	Cold Start(Cache) Increasing block size improves performance, due to improved cache efficiency , but it is still coarsened.
3	16	64	64	2079	This is still a coarsened kernel, but keeping the Grid size the same and increasing the block size will increase the Register Pressure (more number of registers required per thread)
4	32	16	16	1360	Still coarsened: More blocks improves performance.
5	32	32	32	2044	Performance drops slightly, possibly due to occupancy limits
6	32	64	64	2470	Further increase in block size reduces performance
7	64	16	16	1642	More blocks with small threads performs reasonably
8	64	32	32	1799	Slight improvement but not optimal
9	64	64	64	2101	This is not coarsened, but Lesser Occupancy (Maximum number of warps active per second);

Table 1: Performance comparison of 1D grid/block matrix multiplication kernel for matrix of shape 64×64



(a) 64×64 matrix



(b) 100×100 matrix

Figure 2: matmul_1d for 64×64 vs 100×100 matrices

The trend continues still and 100×100 is considerably a similar sized matrix to 64×64 matrix.

Ultimately: The most likely limiter is:

1. LATENCY, if there is no sufficient parallelism(Coarsening);
2. MATH BANDWIDTH, if there is sufficient parallelism and algorithm arithmetic intensity is higher than the GPU $\frac{ops}{byte}$ ratio.
3. MEMORY BANDWIDTH, if there is sufficient parallelism and algorithmic intensity is lower than the GPU $\frac{ops}{byte}$ ratio.

1.2. Matrix-Matrix Multiplication with 2D Grid Parameters

The device kernel code is coarsened(the unit of work per thread is not just one data point, but multiple data points):

```
dim3 num_threads_per_block(block_x, block_y, 1);
dim3 num_blocks_per_grid(grid_x, grid_y, 1);

unsigned int total_threads_x = grid_x * block_x;
unsigned int total_threads_y = grid_y * block_y;

// Ensure at least 1
unsigned int work_per_thread_row = (M + total_threads_y - 1) /
    total_threads_y;
if(work_per_thread_row == 0) work_per_thread_row = 1;
unsigned int work_per_thread_col = (K + total_threads_x - 1) /
    total_threads_x;
if(work_per_thread_col == 0) work_per_thread_col = 1;
```

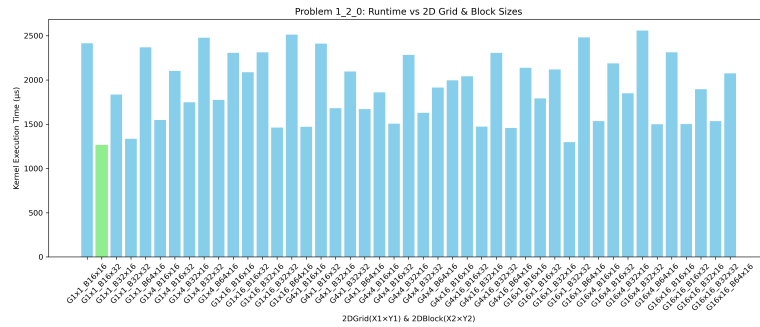


Figure 3: 2D Grid Parameters 64×64 Matrix-Matrix Multiplication

Trial No.	2D Grid Size	2D Block Size	Kernel exec. Time (μ s)	Insights
1	(1,1)	(16,16)	2416	
2	(1,1)	(16,32)	1262	
3	(1,1)	(32,16)	1836	
4	(1,1)	(32,32)	1336	
5	(1,1)	(64,16)	2369	
7	(1,4)	(16,16)	1548	
8	(1,4)	(16,32)	2102	
9	(1,4)	(32,16)	1747	
10	(1,4)	(32,32)	2478	
11	(1,4)	(64,16)	1775	
13	(1,16)	(16,16)	2306	
14	(1,16)	(16,32)	2088	
15	(1,16)	(32,16)	2313	
16	(1,16)	(32,32)	1463	
17	(1,16)	(64,16)	2514	
19	(4,1)	(16,16)	1471	
20	(4,1)	(16,32)	2411	
21	(4,1)	(32,16)	1682	
22	(4,1)	(32,32)	2096	
23	(4,1)	(64,16)	1861	
25	(4,4)	(16,16)	1507	
26	(4,4)	(16,32)	2283	
27	(4,4)	(32,16)	1628	
28	(4,4)	(32,32)	1915	
29	(4,4)	(64,16)	1996	
31	(4,16)	(16,16)	1996	
32	(4,16)	(16,32)	1473	
33	(4,16)	(32,16)	2307	
34	(4,16)	(32,32)	1459	
35	(4,16)	(64,16)	2137	
37	(16,1)	(16,16)	2137	
38	(16,1)	(16,32)	1791	
39	(16,1)	(32,16)	1298	
40	(16,1)	(32,32)	2482	
41	(16,1)	(64,16)	1535	
43	(16,4)	(16,16)	1535	
44	(16,4)	(16,32)	2187	
45	(16,4)	(32,16)	1849	
46	(16,4)	(32,32)	1499	
47	(16,4)	(64,16)	2313	
49	(16,16)	(16,16)	2313	
50	(16,16)	(16,32)	1501	
51	(16,16)	(32,16)	1536	
52	(16,16)	(32,32)	2075	
53	(16,16)	(64,16)	2075	

Table 2: Performance of the matrix-matrix multiplication kernel for various 2D Grid Parameters Combinations

Summary: The performance of the 2D grid/block matrix multiplication kernel depends strongly on both grid and block dimensions. Smaller block sizes underutilize the GPU, while oversized blocks reduce concurrency due to register and memory pressure. Moderate block sizes, combined with enough blocks to fill the streaming multiprocessors, achieve the best latency hiding and throughput. For very small matrices, lower thread counts per block often perform better due to reduced overhead, whereas larger matrices benefit from increased parallelism. The kernel performance is a balance between occupancy, per-thread work, and memory access efficiency.

- For naive matmul with tiny grids:
 - (16,32) wins because each thread does more useful computation, memory accesses align better, and overhead from 1024-thread blocks isn't justified.
 - (32,32) starts to shine only when matrices are large enough (e.g. 1024×1024) where the extra parallelism outweighs per-thread overhead.

2. Tiled Matrix-Matrix Multiplication with Dynamic Shared Memory configuration

```

for (unsigned int phase = 0; phase < num_tiles; ++phase)
{
    unsigned int effective_col_inside_tile = phase * tile_width +
        col_in_tile;

    if (tid_y < M && effective_col_inside_tile < N) {
        TILE_A[row_in_tile * tile_width + col_in_tile] =
            d_A[tid_y * N + effective_col_inside_tile];
        atomicAdd(&d_gmem_reads, 1ULL);
    } else {
        TILE_A[row_in_tile * tile_width + col_in_tile] = 0;
    }
    atomicAdd(&d_smem_writes, 1ULL);

    unsigned int effective_row_inside_tile = phase * tile_width +
        row_in_tile;

    if (tid_x < K && effective_row_inside_tile < N) {
        TILE_B[row_in_tile * tile_width + col_in_tile] =
            d_B[effective_row_inside_tile * K + tid_x];
        atomicAdd(&d_gmem_reads, 1ULL);
    } else {
        TILE_B[row_in_tile * tile_width + col_in_tile] = 0;
    }
    atomicAdd(&d_smem_writes, 1ULL);

    __syncthreads(); // To Avoid Data Race

    for (unsigned int k = 0; k < tile_width; ++k) {
        tmp += TILE_A[row_in_tile * tile_width + k] *
            TILE_B[k * tile_width + col_in_tile];
        atomicAdd(&d_smem_reads, 2ULL); // one read from TILE_A + one
            from TILE_B
    }

    __syncthreads(); // To make the right update visible
}

if (tid_y < M && tid_x < K) {
    d_C[tid_y * K + tid_x] = tmp;
    atomicAdd(&d_gmem_writes, 1ULL);
}

```

The `__syncthreads()` barrier is expensive in time(waiting or wasting the clock cycles of the machine) and it won't scale well for larger tile sizes for the same input matrix size.

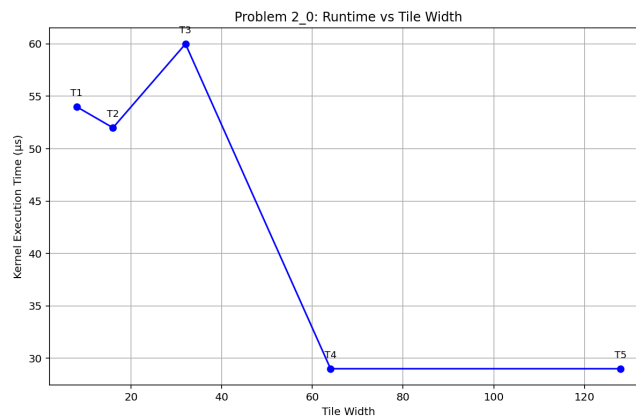


Figure 4: 64×64 Tiled Matrix-Matrix Multiplication Time

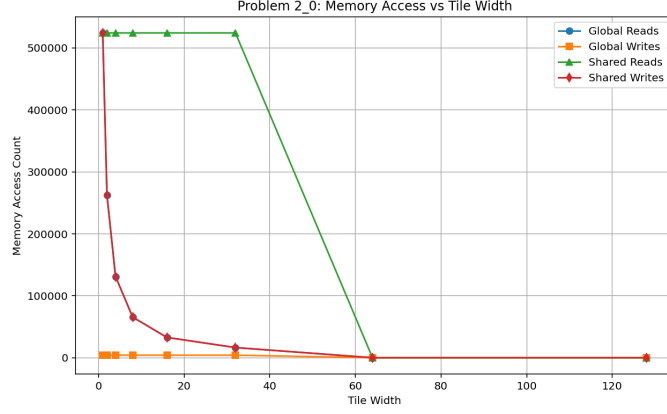


Figure 5: 64×64 Tiled Matrix-Matrix Multiplication Memory

SL No.	Tile Size	Global Reads	Global Writes	Shared Reads	Shared Writes	Kernel exec. Time (μ s)	Insights
1	1	524288	4096	524288	524288	830	No tiling; huge redundant global accesses, very slow
2	2	262144	4096	524288	262144	171	Some reuse; reduced global reads, big improvement
3	4	131072	4096	524288	131072	65	Better reuse; optimal region begins
4	8	65536	4096	524288	65536	71	Good balance, stable execution time
5	16	32768	4096	524288	32768	100	Fewer global reads, but overhead increases
6	32	16384	4096	524288	16384	130	Larger tile, diminishing returns; more sync cost
7	64	0	0	0	0	-	Block Size exceeds
8	128	0	0	0	0	-	Block Size exceeds

Table 3: Performance of shared-memory tiled matrix multiplication for 64×64 matrices with varying tile sizes.

Grid Dimensions Calculation using the Tile Width

```

dim3 block(tile_width, tile_width);
dim3 grid((K + tile_width - 1) / tile_width,
          (M + tile_width - 1) / tile_width);
size_t shared_size_in_bytes =
    2 * tile_width * (tile_width + 1) * sizeof(int32_t); // To avoid Shared Memory Bank Conflict
matrix_multiplication_tiled_dkernel<<<grid, block, shared_size_in_bytes>>>(
    d_C, d_A, d_B, M, N, K, tile_width);

```

Partial Result Computations $\rightarrow$ Data Reuse: The figure 5 shows that the shared memory accesses are significantly higher than the global memory accesses.

Since the operation here is just multiply and add, there is not much math per instruction per input data point involved here, because the GPU hardware is optimized to do FMA.

3. Matrix Transpose with 1D Grid Parameters

```
// 1D coarsened transpose kernel
__global__ void transpose_1d_kernel(int32_t* A_T, const int32_t* A,
                                   unsigned int num_rows, unsigned int
                                   num_cols,
                                   int work_per_thread)
{
    unsigned int tid = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned int total = num_rows * num_cols;

    for(int w = 0; w < work_per_thread; ++w){
        unsigned int idx = tid + w * blockDim.x * blockDim.x;
        if(idx >= total) break;

        unsigned int row = idx / num_cols;
        unsigned int col = idx % num_cols;

        A_T[col * num_rows + row] = A[row * num_cols + col];
    }
}
```

Memory Access Pattern. Each thread with global ID

$$tid = blockIdx.x \cdot blockDim.x + threadIdx.x$$

processes multiple elements separated by a stride

$$\Delta = blockDim.x \cdot blockDim.x.$$

For the w -th iteration, the linear index is

$$idx = tid + w \cdot \Delta.$$

This corresponds to a matrix element at

$$row = \left\lfloor \frac{idx}{num_cols} \right\rfloor, \quad col = idx \bmod num_cols.$$

- **Read access (coalesced):**

$$A[row \cdot num_cols + col],$$

which corresponds to accessing consecutive columns in the same row. Thus, consecutive threads in a warp fetch consecutive elements from memory.

- **Write access (non-coalesced):**

$$A^T[col \cdot num_rows + row],$$

which corresponds to writing down columns of the output matrix. Therefore, consecutive threads in a warp write to memory locations separated by a stride of `num_rows`.

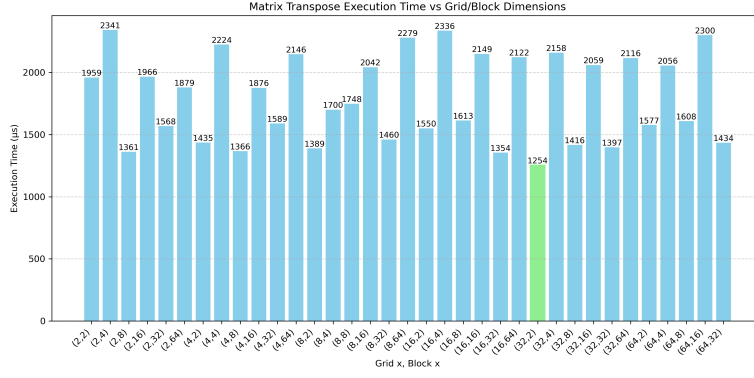


Figure 6: 64×64 Matrix Transpose Kernel Performance with respect to the 1D Parameters i.e., num_blocks_per_grid(Grid_x), num_threads_per_block (Block_x)

SL No.	Thread Blocks (grid_x)	Block Size (block_x)	Kernel exec. Time (µs)	Insights
1	2	2	1959	Very few threads, high work per thread → higher latency
2	2	4	2341	Slight increase in threads, but overhead dominates
3	2	8	1361	Better parallelism, improved latency
4	2	16	1966	Performance drops due to divergence/occupancy issues
5	2	32	1568	More balanced, moderate performance
6	2	64	1879	Larger block increases imbalance, slower
7	4	2	1435	More blocks → better scheduling, faster
8	4	4	2224	Still suboptimal, memory effects visible
9	4	8	1366	Good balance, one of the faster configs
10	4	16	1876	More threads but efficiency decreases
11	4	32	1589	Balanced, moderate speed
12	4	64	2146	Performance degraded with too many threads per block
13	8	2	1389	Good scaling with higher grid_x
14	8	4	1700	Slight slowdown, more scheduling overhead
15	8	8	1748	Similar slowdown, memory access pattern matters
16	8	16	2042	Overhead visible again
17	8	32	1460	Stable performance, moderate speed
18	8	64	2279	Too many threads per block → worse perf
19	16	2	1550	Many small blocks, decent balance
20	16	4	2336	Overhead dominates, slower
21	16	8	1613	Moderate performance
22	16	16	2149	Less efficient
23	16	32	1354	One of the best results (good occupancy + balance)
24	16	64	2122	Again slower at larger block size
25	32	2	1254	Best performance observed — many small blocks, good scheduling
26	32	4	2158	Sharp slowdown, overhead heavy
27	32	8	1416	Fast, comparable to best cases
28	32	16	2059	Slower at larger block size
29	32	32	1397	Very good performance, close to best
30	32	64	2116	Slower due to memory inefficiency
31	64	2	1577	Decent performance, but not best
32	64	4	2056	Higher overhead, slower
33	64	8	1608	Moderate, acceptable perf
34	64	16	2300	Very slow, too much scheduling overhead
35	64	32	1434	Good, near-optimal compared to best case

Table 4: Performance comparison of the 1D coarsened transpose kernel across grid and block size combinations.

All of these kernel launches are coarsened kernels.

4. Tiled Matrix Transpose

Effect due to Warp Alignment and Memory Access Patterns

```
// -----  
// 4) load from input into shared memory  
// -----  
if (global_row_in < num_rows && global_col_in < num_cols) {  
    unsigned int idx_in = global_row_in * num_cols + global_col_in;  
    TILE[local_row * stride + local_col] = d_input[idx_in];  
} else {  
    TILE[local_row * stride + local_col] = 0; // padding for out-of-  
        bounds  
}  
  
__syncthreads();  
  
// -----  
// 5) compute block origin in the output matrix  
// (notice: blockIdx.x <-> blockIdx.y swapped)  
// -----  
unsigned int block_start_col_out = blockIdx.y * blockDim.y; //  
    becomes columns in output  
unsigned int block_start_row_out = blockIdx.x * blockDim.x; //  
    becomes rows in output  
  
// -----  
// 6) compute global coords in the output matrix  
// -----  
unsigned int global_col_out = block_start_col_out + local_col; //  
    [0 .. num_rows-1]  
unsigned int global_row_out = block_start_row_out + local_row; //  
    [0 .. num_cols-1]  
  
// -----  
// 7) store from shared memory transposed  
// output is [num_cols x num_rows]  
// -----  
if (global_row_out < num_cols && global_col_out < num_rows) {  
    unsigned int idx_out = global_row_out * num_rows +  
        global_col_out; // row-major: out_cols = num_rows  
    d_output[idx_out] = TILE[local_col * stride + local_row];  
}
```

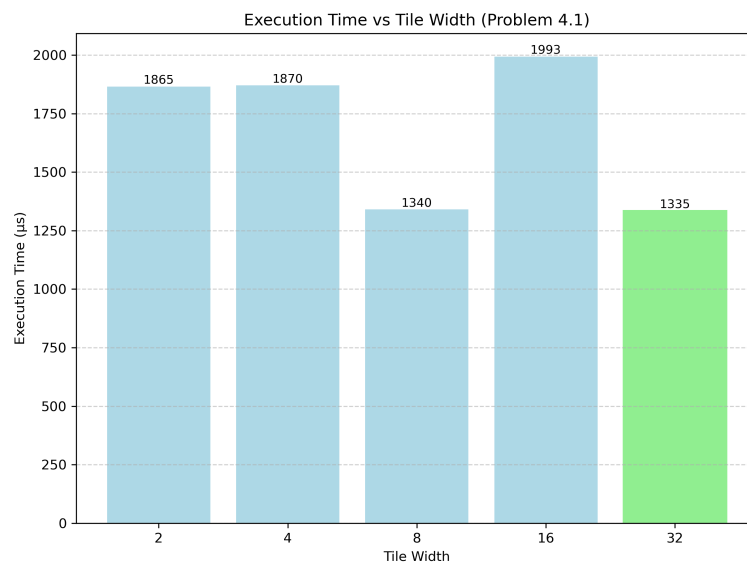


Figure 7: 64×64 Matrix

SL No.	Tile Size	Kernel exec. Time (μ s)	Insights
1	2	1865	Small tile size, baseline performance
2	4	1870	Slight increase, minimal impact on performance
3	8	1340	4 transactions per warp, but better GPU utilization
4	16	1993	2 transactions per warp;
5	32	1335	1 transaction per warp, ideal to warp size(32)

Table 5: Performance comparison of tiled transpose kernel with respect to change in the tile width

Why is the tile size 16 taking so much time?

Tile Size = 16 Each block = $16 \times 16 = 256$ threads = 8 warps. In the load phase, warps are actually split across rows:

- Warp 0 = threads (row 0, col 0..15) + (row 1, col 0..15)
- Warp 1 = threads (row 2, col 0..15) + (row 3, col 0..15)
- ... etc.

So each warp is handling two different rows. Each half-warp (16 threads) fetches 16 contiguous elements $\rightarrow$ that's fine. But since the row length is 16, the warp fetch ends up being two disjoint 64-byte segments, not a single 128-byte segment.

Result: the hardware issues two memory transactions per warp.

This doubles the memory traffic compared to tile = 32.

Tiled vs Naive Transpose at Scale

For small matrices (like 64×64), the naive/coarsened transpose kernel can be as fast or even faster than the tiled version due to low memory traffic. However, as the matrix size increases, the tiled transpose kernel scales much better because it ensures coalesced global memory access and better bandwidth utilization, leading to consistently superior performance compared to the naive approach.

Tiled Matrix Transpose: Memory Access Pattern

1. Load from A $\rightarrow$ Shared Memory (Read phase)

Tile in A (row-major):

```
Global memory (A) row-major:

Row 0: A00 A01 A02 A03 ...
Row 1: A10 A11 A12 A13 ...
Row 2: A20 A21 A22 A23 ...
Row 3: A30 A31 A32 A33 ...
```

Warp threads read a row each (coalesced):

```
Thread(0,0) -> A00
Thread(0,1) -> A01
Thread(0,2) -> A02
Thread(0,3) -> A03 <-- consecutive memory
```

```
Thread(1,0) -> A10
Thread(1,1) -> A11
...
```

⇒ All threads in a warp read consecutive addresses (row-major). **Reads are coalesced.**

2. Shared memory tile (before transpose)

```
SMEM before transpose:

[ A00 A01 A02 A03 ]
[ A10 A11 A12 A13 ]
[ A20 A21 A22 A23 ]
[ A30 A31 A32 A33 ]
```

3. Shared memory tile (after transpose)

Each thread swaps indices locally in SMEM:

```
SMEM after transpose:

[ A00 A10 A20 A30 ]
[ A01 A11 A21 A31 ]
[ A02 A12 A22 A32 ]
[ A03 A13 A23 A33 ]
```

4. Store to B → Global Memory (Write phase)

Now threads write the transposed tile back to B in row-major:

```
Global memory (B) row-major:

Row 0: A00 A10 A20 A30
Row 1: A01 A11 A21 A31
Row 2: A02 A12 A22 A32
Row 3: A03 A13 A23 A33
```

Writes pattern:

```
Thread(0,0) -> B00
Thread(1,0) -> B01
Thread(2,0) -> B02
Thread(3,0) -> B03 <-- consecutive in memory!
```

⇒ Each warp now writes a row of B (not a column), so addresses are consecutive. **Writes are coalesced.**

Tiled Versions: Matrix-Matrix Multiplication vs Matrix Transpose

The device kernel execution time difference for tiled matrix transpose and tiled matrix multiplication is significantly large. The **Reads** are **coalesced** but the **writes** are **not**, in tiled matrix transpose, unlike the tiled matrix-matrix multiplication, in which there is a lot **Arithmetic Intensity** ⇒ **Compute-Bound**, which can lead to the overlap of the computation and communication, and there is a lot **Data-Reuse**.

Memory Ordering is also one of the factors affecting this significant performance gap

References

- [1] Robert Crovella. About latency (very basics), 2014.
- [2] Robert Crovella. Coalescing, 2019.